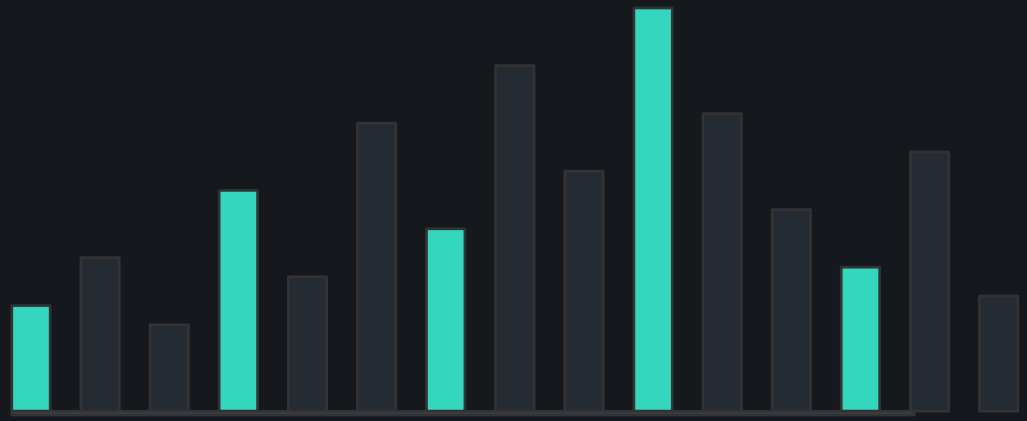
EDUMATCHER · OPERATOR TRAINING

Configuring a Trading System



From an empty file to a running exchange — what every setting means, why it matters, and how to prove it is correct before anyone trades.

A beginner's guide for new exchange operators

WHY THIS MATTERS

Configuration is the exchange

The matching engine ships with no opinions. Which instruments exist, who may connect, when the market opens, how far a price may move before it is rejected — none of that is in the code. It is all in one file. Change the file and you have changed the exchange.

1
shared reference-data file

20
top-level sections

2
sections that are mandatory

10
processes that read from it

The uncomfortable truth

- **Silent.** A missing config file does not stop the engine — it starts wide open.
- **Partial.** A typo in one section is invisible to the process that never reads it.
- **Late.** The loader reports only the first error, then aborts.
- **Costly.** A wrong tick size or missing collar is discovered by a bad trade.

ROADMAP

What this session covers

Nine questions, answered in order. No prior knowledge assumed.

01
What is configuration?
The mental model

02
What must be configured
The category map

03
How to create it
By hand, CLI, or GUI

04
How to verify it
pm-cverifier

05
Why a compiled artifact
YAML in, JSON out

06
Every top-level section
Field by field

07
The minimal configuration
The smallest thing that works

08
Who reads the config
Direct and indirect readers

09
A workflow that works
Setting up a new exchange

The last few slides are a technical reference you can come back to.

THE WHOLE STORY

The 60-second mental model

Everything in this deck is one of these five moves.

AUTHOR

Write it

engine_config.yaml — the file you edit and keep in version control

VERIFY

Check it

pm-cverifier reads every section and reports every problem at once

COMPILE

Deploy it

pm-config-deploy resolves defaults and installs engine_config.json

RUN

Start it

pm-engine first, then scheduler, gateways and observers

OPERATE

Prove it

Query the running exchange to confirm it matches your intent

THE ONE RULE TO REMEMBER

You author YAML, but no running process ever reads it. Every process reads the compiled JSON artifact produced by pm-config-deploy. Editing YAML without deploying changes nothing.

Sections 03, 04, 05 and 09 of this deck each expand one of these five moves.

SECTION 01

What is configuration of a trading system?

01

Before the settings, the concepts. What an exchange actually is, and which parts of it are decided by a file rather than by code.

The exchange, assembled

Code vs. configuration

Unrestricted mode

The vocabulary

FIRST PRINCIPLES

What a trading system is made of

An exchange is not one program. It is a matching engine surrounded by gateways, and a shared file that tells them all who and what is allowed.

Participants

Traders, market makers and operators connect through a gateway

Post-trade

Fills, drop-copy and audit feeds for clearing

pm-engine

The matching engine
The single writer of the order book

Market data

Books, trades and indices published out to subscribers

Observers

Viewers, tickers, boards and P&L tools

Every one of these processes learns what it may do from the same configuration.

DEFINITION

Configuration is reference data

In exchange language, "reference data" is the set of facts about the market that must be true before the first order arrives.

It answers four questions

- **What trades?** The symbol universe — the instrument allowlist.
- **Who may connect?** The gateway allowlist, and each one's role.
- **When is the market open?** Sessions, auctions and the daily schedule.
- **What is not allowed?** Collars, circuit breakers and obligations.

engine_config.yaml

```yaml
symbols:
  AAPL:
    tick_decimals: 2
    last_buy_price: 149.90
    last_sell_price: 150.10
gateways:
  alf:
    - id: TRADER01
```

READ THIS AS

One instrument called AAPL priced in cents around 150, and one participant called TRADER01 who is allowed to connect. That is already a working exchange.

BOUNDARIES

What configuration decides — and what it never touches

A common beginner question: if it is in the file, can I change it? Here is the line.

Configuration decides

- **Universe** which symbols exist and their price precision
- **Access** which gateway IDs may connect, and their roles
- **Policy** collars, circuit-breaker ladders, MM obligations
- **Rhythm** session phases, auction times, holiday calendar
- **Plumbing** ports, timeouts, queue limits for every gateway

Code decides

- **Matching** price-time priority and the auction uncross
- **Order types** what LIMIT, STOP or ICEBERG actually do
- **Protocols** the ALF, BALF, CALF and RALF message formats
- **Transport** the ZeroMQ socket pattern between processes
- **Persistence** which state survives a restart

RULE OF THUMB

Configuration describes your market. Code implements the mechanics of any market. If a change would alter how matching works, it is not a config change.

THE FAILURE YOU WILL NOT NOTICE

No config file? The engine still starts.

This surprises everyone. A missing configuration is not an error — it is a mode.

Unrestricted mode means

No symbol allowlist · No gateway allowlist · No risk levels · No seeded last prices · No market-maker quotes · No startup combos · No outstanding-share metadata · Sessions disabled

Books are created on demand

Any gateway ID can connect. An order for any symbol is accepted, and an order book for it is created on first use. Nothing is rejected, because nothing was ever declared.

Convenient for a first experiment. Dangerous as a default.

PROCESS	MISSING DEFAULT CONFIG	MISSING EXPLICIT --CONFIG
pm-engine	Starts unrestricted	Starts unrestricted for that path
pm-scheduler	Uses the built-in schedule	Fatal error

So "the engine started" is never proof that your configuration was loaded.

GLOSSARY

Eight words you need before we go further

Every one of these appears in the configuration file.

Symbol

A tradable instrument. AAPL, MSFT. Always stored upper-case.

Gateway

A permitted connection identity — a trader, a market maker, or an operator console.

Role

What a gateway may do: TRADER, MARKET_MAKER or ADMIN.

Tick

The smallest price increment. tick_decimals: 2 means cents.

Collar

A price band. Orders outside it are rejected — fat-finger protection.

Circuit breaker

An automatic trading halt when a price moves too far, too fast.

Session

A phase of the trading day: pre-open, auction, continuous, closed.

Seed quote

A resting bid and ask injected at startup so the book is not empty.

SECTION 02

What categories must be specified?

02

Twenty top-level keys, but only two are required. Here is how to think about the rest — grouped by the job they do rather than by their order in the file.

Required vs optional

Six categories

The full key map

Default ports

THE WHOLE SURFACE

Twenty top-level keys, six jobs

Colour tells you which category a key belongs to. Two keys carry a solid border: those are the only mandatory ones.

IDENTITY & UNIVERSE	**symbols** · **gateways.alf**
ENGINE BEHAVIOUR	sessions_enabled · enforce_collars · enforce_circuit_breakers · snapshot_interval_sec
RISK POLICY	risk_controls · circuit_breaker_defaults
LIQUIDITY	mm_obligation_defaults · market_maker_combos
MARKET RHYTHM & DATA	schedule · country · indices
CONNECTIVITY BLOCKS	alf_gateway · balf_gateway · market_data_gateway · post_trade_gateway · dc_gateway · api_gat · log_server

READ THE MAP THIS WAY

Start at the top and stop as soon as your exchange works. Rows one and two get you trading; rows three to five make the market realistic; row six only matters once external clients need to connect.

THE FLOOR

Only two things are required

Everything else in the file has a default. These two do not — omit either and the engine refuses to start.

1 `symbols`

A mapping of symbol name to its settings. This is the instrument allowlist: an order for anything not listed here is rejected with "Symbol not configured".

MUST be a mapping. If it is anything else, the engine exits with a fatal config error.

2 `gateways.alf`

A list with at least one entry. This is the participant allowlist. A gateway ID that is not here can still attempt to connect — the engine simply ignores it, and the client times out.

MUST be a list with one or more entries. IDs are unique and stored upper-case.

WHY THE SILENT IGNORE MATTERS

An unknown gateway ID produces no error on the engine side at all. The only symptom is a client that says "Gateway authentication timed out". Nine times out of ten that is a missing entry in gateways.alf.

CATEGORY 1 OF 6

Identity & universe

Who exists and what trades. Nothing else in the file makes sense until these are right.

`symbols` `gateways.alf`

- **Symbol name** — The key of the symbols mapping. Upper-cased on load — aapl and AAPL are the same instrument.
- **tick_decimals** — Price precision, 0 to 8, default 2. Two decimals means prices move in cents. Get this wrong and every price displays wrong.
- **last_buy_price / last_sell_price** — Seed reference prices. On day one there is no trade history, so collars and circuit breakers have nothing to anchor to unless you supply these.
- **outstanding_shares** — Share count. Optional — until the symbol joins a cap-weighted index, where it becomes required.
- **id / role** — The gateway identity and what it may do. The ID must exactly match what the client passes on the command line.

WHY A BEGINNER SHOULD CARE

A beginner's exchange needs exactly this: a handful of symbols with sane prices, and one gateway ID per person in the room.

CATEGORY 2 OF 6

Engine behaviour flags

Four booleans and one number that change how the engine behaves globally. All default to safe values.

`sessions_enabled` `enforce_collars` `enforce_circuit_breakers` `snapshot_interval_sec`

- **sessions_enabled** — Default true. When true the engine starts CLOSED and waits to be driven through the trading day. When false it starts and stays in CONTINUOUS.
- **enforce_collars** — Default true. The master switch for price-band rejection. Turning it off does not delete your collar settings — it just stops applying them.
- **enforce_circuit_breakers** — Default true. The master switch for volatility halts.
- **snapshot_interval_sec** — Default 0.5. The minimum gap between published order-book snapshots for a busy symbol. Lower means more data and more load.

WHY A BEGINNER SHOULD CARE

The classic beginner trap: sessions_enabled is true by default, so the engine starts CLOSED. If nobody runs the scheduler, the market never opens and no order will match.

CATEGORY 3 OF 6

Risk policy

The two mechanisms that stop a bad price from becoming a bad trade. Both are on by default.

`risk_controls` `circuit_breaker_defaults`

- **Collars** — A price band around a reference price. static_band_pct anchors to a session reference (default 20%); dynamic_band_pct tracks near-live prices (default 2%). Orders outside are rejected.
- **Risk levels** — Named bundles of collar settings — CORE, HIGH_BETA — that symbols reference by name, so you set the policy once and apply it to many instruments.
- **default_level** — The level applied to any symbol that does not name its own. Must be a level that actually exists.

- **Circuit-breaker ladder** — Levels L1, L2, L3, each with a price shift that triggers it and a halt duration. Built-in ladder: 7% for 5 min, 13% for 15 min, 20% for the rest of the day.
- **Reopening (ACE)** — Automated Corridor Expansion — how the market restarts after a halt, widening the allowed corridor in stages until it can reopen.

WHY A BEGINNER SHOULD CARE

Without collars, a student who types 15000 instead of 150.00 prints a trade at a hundred times the market. Leave both mechanisms on.

CATEGORY 4 OF 6

Liquidity

An empty order book teaches nobody anything. These settings put resting prices in the book before the first participant connects.

`mm_obligation_defaults` `market_maker_quotes` `market_maker_combos`

- **market_maker_quotes** — Per symbol: a bid price, an ask price and sizes, attributed to a MARKET_MAKER gateway. The engine injects them when that gateway authenticates.
- **seed_once** — Default true. Skip injection if the engine already has prior book state for the symbol, so a restart does not double-seed.
- **mm_obligation_defaults** — Exchange-wide market-maker duties: whether obligations are enforced at all, the widest allowed spread in ticks (default 10) and the minimum size on each side (default 100).
- **market_maker_combos** — Multi-leg seed orders — two to ten legs — placed as a single all-or-none package at startup. Spreads, pairs and hedged entries.

WHY A BEGINNER SHOULD CARE

If a MARKET_MAKER gateway exists, the spec then requires every symbol to define at least one quote. Adding the role is what triggers the requirement.

CATEGORY 5 OF 6

Market rhythm & analytics

When the market is open, which days it skips, and what is calculated from it.

`schedule` `country` `indices`

- **schedule** — Five wall-clock times: pre_open 09:00, opening_auction_start 09:25, continuous_start 09:30, closing_auction_start 16:00, closing_auction_end 16:05. They must be strictly increasing.
- **country** — Default "Sweden". A country name or ISO code used to look up bank holidays. The scheduler treats weekends and holidays as non-trading days.
- **indices** — Up to five cap-weighted indices. Each names its constituents, a base value (default 1000) and how often it publishes (default every second).
- **Constituent rule** — Every constituent must be a configured symbol that also defines outstanding_shares — the weight cannot be computed without it.

WHY A BEGINNER SHOULD CARE

country is a top-level key, a sibling of schedule — not nested inside it. It is also the one field where a bad value produces a warning instead of a failure.

CATEGORY 6 OF 6

Connectivity blocks

Seven optional blocks, one per external-facing service. Each is read only by its own process — the engine ignores all of them.

`alf_gateway` `balf_gateway` `market_data_gateway` `post_trade_gateway` `dc_gateway` `api_gateways` `log_server`

- **The same shape every time** — Each block carries a bind address, a port, heartbeat and idle timeouts, connection limits and queue caps. Learn one and you have learned all seven.
- **alf_gateway / balf_gateway** — Order entry, text and binary. Both also read gateways.alf for the identity and role of each session.
- **market_data_gateway** — The CALF feed: top-of-book, trades and index values, with a replay window and a depth level count.
- **post_trade_gateway / dc_gateway** — Fills and drop-copy for clearing and audit. Neither has an enabled key — configuring it is enabling it.
- **api_gateways / log_server** — Named REST and WebSocket instances with API credentials, and the central log collector with its retention settings.

WHY A BEGINNER SHOULD CARE

You need none of these for a classroom exchange driven by consoles. Add them the day something outside the room needs to connect.

DECISION AID

Do I actually need this section?

A practical reading of the twenty keys for someone standing up their first exchange.

SECTION	STATUS	SKIP IT IF…
symbols	REQUIRED	Never. Without it the engine will not start.
gateways.alf	REQUIRED	Never. Without it nobody can connect.
schedule + sessions_enabled	Recommended	You are happy running always-open: set sessions_enabled to false.
market_maker_quotes	Recommended	You will place the first orders manually and do not mind an empty book.
risk_controls	Optional	Default collar bands of 20% static and 2% dynamic are fine for you.
circuit_breaker_defaults	Optional	The built-in 7/13/20 percent ladder suits your scenario.
mm_obligation_defaults	Optional	You are not grading market makers on quote quality.
indices	Optional	You do not need an index published.
market_maker_combos	Optional	You are not teaching multi-leg orders yet.
The seven gateway blocks	Optional	Everyone connects with the bundled console tools.

Start with the two required sections plus a schedule and a seed quote. Add the rest when you have a reason.

CONNECTIVITY AT A GLANCE

Every default port in one picture

If two blocks end up on the same port, nothing starts. This is the map to check against.

5555
Engine PULL
orders in

5556
Engine PUB
events out

5557
Engine PUB
drop copy

5560
balf_gateway
binary order entry

5565
alf_gateway
text order entry

5570
market_data_gateway
CALF market data

5580
post_trade_gateway
RALF post-trade

5590
dc_gateway
drop-copy relay

5600
log_server
LALF log collector

5601
log_server pub
log distribution

5602
log_server pull
subscriber control

8080
api_gateways
REST / WebSocket

COLLISION CHECK

The verifier compares effective ports — the value you wrote, or the default if you wrote nothing. Setting log_server.pub_port to 5600 collides with the log server's own default listen port even though you never typed 5600 twice.

SECTION 03

How can a configuration be created?

03

Three roads lead to the same file. Pick the one that matches how you work — and know that the file, not the tool, is what matters.

By hand

pm-config-gen

config-gui

Choosing well

OVERVIEW

Three ways to produce the same file

All three emit an engine_config.yaml in exactly the same format. Nothing downstream can tell which one you used.

By hand

A text editor

- **Strength** Total control. Comments stay where you put them. Diffs are meaningful in version control.
- **Cost** Nothing catches a mistake until you run something. Easy to miss a required field.
- **Best for** Small edits to a file you already understand.

pm-config-gen

One command

- **Strength** Repeatable and scriptable. Consistent defaults every time. Ideal inside CI.
- **Cost** Concise flags only reach so far — complex risk or index setups still need editing afterwards.
- **Best for** Bootstrapping a new class or demo from scratch.

config-gui

A web application

- **Strength** Forms with inline help, live cross-field validation, and an export gate that blocks broken files.
- **Cost** Needs a container runtime. Inline comments in an imported file are not preserved.
- **Best for** Learning the schema, or building a full environment visually.

COMMON PRACTICE

Generate the skeleton with pm-config-gen or the GUI, then hand-edit the details. Verify whichever way you got there.

PATH A

Writing it by hand

The file is YAML: indentation is structure, and a mapping is a set of key/value pairs. Two rules cause most beginner failures.

engine_config.yaml

```yaml
gateways:
  alf:                        # a LIST — note the dashes
    - id: TRADER01
      description: First trader
    - id: MM01
      role: MARKET_MAKER
    - id: GW_ADMIN
      role: ADMIN

symbols:                      # a MAPPING — note no dashes
  AAPL:
    tick_decimals: 2
    last_buy_price: 149.90
```

Rule 1 — shape matters

gateways.alf must be a list. symbols must be a mapping. Swap them and the engine exits with a fatal config error naming the offending key.

Rule 2 — quote your times

Schedule values must be quoted strings: "09:30", not 09:30. Unquoted, YAML reads it as a sexagesimal number and the schedule silently becomes nonsense.

PATH B

pm-config-gen — bootstrap in one line

Built for operators and instructors who want a new session running without writing large YAML blocks by hand.

terminal

```
pm-config-gen \
    --symbols AAPL MSFT TSLA \
    --gateways TRADER01 MM01:MARKET_MAKER GW_ADMIN:ADMIN \
    --output engine_config.yaml
```

Read the gateway argument as ID:ROLE. Omit the role and the gateway is a plain TRADER.

Use it when

You are creating a new class or demo config from scratch, want consistent defaults, or need repeatable generation inside a script.

What you still do

Fill in the market-maker bid and ask prices before starting the engine. Generated quote blocks arrive as stubs, not prices.

The companion flag

Ask it to emit the full commented reference block, so the generated file doubles as documentation of every field you did not set.

PATH C

config-gui — the Config Builder

A browser application for creating, importing, editing, validating and exporting the same engine_config.yaml. A companion to the CLI, not a replacement.

Live cross-field validation

Undefined risk levels, port collisions and out-of-order schedules are flagged as you type, not after you run something.

Progressive disclosure

Beginners see a handful of fields. Experts see everything. The same file underneath.

Full round-trip import

Load an existing file, edit it visually, export it back. Sections it does not model are preserved verbatim.

the quickest way to run it

```
podman load --input dist/edumatcher-config-gui-<VER>.tar.gz
podman run -p 8080:8080 edumatcher-config-gui:<VER>

# then open  http://localhost:8080/
```

The correctness guarantee

A golden-file test pipes generated configs through the engine's own loader. Anything the GUI exports is guaranteed to load — that check is what keeps two implementations of the same format honest.

CONFIG-GUI

A tour of the interface

Three panes that never move, so you always know where you are and what is wrong.

Navigation

The tab list. Each tab carries a status glyph — valid, warning, error, or untouched — so the health of the whole config is visible at a glance.

Active panel

The form for the selected tab. Mandatory fields carry a red marker; optional fields show their default as ghost text until you set them. Every non-obvious field has a help popover naming the equivalent CLI flag.

Diagnostics drawer

Always visible. Lists every issue across the whole configuration — not just this tab — each with a jump action that navigates to and highlights the offending field.

PLUS A TOP BAR

Import an existing file, start a blank draft, toggle the theme, and save. Your draft is autosaved continuously in the browser, so a refresh or a crash never loses work.

Personas — see only what you need

One global setting controls how much of the field surface is visible. It is a view filter, never a data filter: switching never discards a value.

Beginner

A first-time student standing up a classroom exchange.

Show only what a minimal, safe exchange needs. Everything else takes a safe default.

TABS

Basics · Sessions & Schedule · Market Maker · Review & Export

Intermediate

A teaching assistant or instructor tuning a scenario.

Adds risk levels, circuit-breaker tuning, per-symbol overrides, indices and the common gateways.

TABS

+ Risk & Collars · Circuit Breakers · Symbols · Indices · Auxiliary Gateways

Expert

A course author building the full environment.

Everything — combos, per-gateway MM overrides, BALF and API gateways, engine tuning, deterministic seeding.

TABS

+ Combos · Engine Tuning

The Market Maker tab appears at any persona — but only once at least one gateway carries the MARKET_MAKER role.

CONFIG-GUI

Adding a symbol is an IPO

The GUI does not ask for a symbol name. It opens a listing dialog — because on the very first tick the engine has no trade history to anchor to.

List a new symbol (IPO)

- **Symbol name** upper-cased automatically, must be unique
- **Reference price** the opening mid — derives last buy/sell prices and the collar's day-one anchor
- **Tick decimals** price precision, 0 to 8
- **Outstanding shares** required if the symbol will join an index
- **Opening spread + quote size** auto-derives the market maker's first bid and ask around the reference price

Why one number drives five

Collar reference, circuit-breaker reference, last buy price, last sell price and the opening quote must all agree. Capturing them separately is how they drift apart. The dialog captures one price and derives the rest.

The Symbol Overview

A read-only view of the effective values — what the engine will actually use after every inheritance and merge rule — with each value tagged by where it came from. Your last sanity check before export.

Seeding the book: two routes and a trap

Before the engine starts there must be resting quotes, or the first participant sees an empty book.

PRECISE

Option A — explicit quotes

Enter a specific bid price, ask price and sizes for each symbol and market-maker pair. Highest precedence. Use when different symbols need different opening prices.

FAST

Option B — mid-range seeding

Set one price range on the Market Maker tab. The builder computes a single midpoint and places a bid one tick below and an ask one tick above.

BLOCKED

Fallback — a null stub

Neither supplied? The builder emits an empty quote. The review panel flags it and export is blocked until you resolve it.

THE TRAP IN OPTION B

The range is averaged into one value and applied to every symbol — it is not distributed across them. A range of 200 to 220 gives every instrument a midpoint of 210, so AAPL and TSLA both open at 209.99 bid, 210.01 ask. For different opening prices per symbol, use explicit quotes.

SEED ONCE

On by default: the engine injects a seed quote only if it has no prior book state for that symbol, so a restart with existing history does not re-inject it.

Live diagnostics and the export gate

Every rule runs on each change, and again before export. Results appear in three coordinated places at once.

SEE IT

Field markers

A coloured border, an inline icon and a tooltip on the offending field itself

SCAN IT

Tab glyphs

Each tab shows the worst severity it contains, so nothing hides behind a closed tab

FIX IT

Diagnostics drawer

A global issue list with a jump action that navigates to and highlights the linked fields

What it checks

The same rules the command-line verifier applies — undefined risk levels, a lone gateway, no admin gateway, disabled risk controls, port collisions, out-of-order schedules — plus structural checks on index constituents, combo legs and obligation symbols.

The export gate

Download is disabled while any error-severity issue exists anywhere in the configuration. Warnings do not block export, but they require a one-time acknowledgement — you cannot ship a broken file by accident.

Importing an existing configuration

You are rarely starting from nothing. Import maps an existing file into the form and opens the review tab first, reporting the health of what you loaded.

1
Import
Load your engine_config.yaml

>

2
Review
The GUI reports every issue it found in the file as loaded

>

3
Edit
Change what you need through the forms

>

4
Export
Download the regenerated file

Nothing is silently dropped

Anything the GUI does not model — a hand-added section, or a field from a newer schema — is preserved verbatim in a per-section passthrough and re-emitted unchanged. A banner marks any tab carrying such content.

One thing is: your comments

The GUI is a structural editor, not a text editor. It regenerates the header and comment block fresh, but inline comments from the imported file are not round-tripped. If your comments are the documentation, keep the authored file as the source of truth.

SECTION 04

How can you verify a configuration?

04

One read-only command that reads the entire file, applies four layers of checks, and tells you in plain English what your configuration actually does.

Why it exists

Four layers

Reading the report

The risk summary

MOTIVATION

Why a verifier exists at all

The engine's own loader is a poor teacher. It was built to load a file, not to explain one.

Without it: the guessing loop

1. Edit the file
2. Start the engine
3. Read one terse error
4. Fix that one thing
5. Start the engine again

With it: one pass, everything

- **Every syntax error** that would stop the file being parsed
- **Every hard error** the loader would raise — and more besides
- **Completeness gaps** settings that are missing but probably should not be
- **Advisories** valid settings that are likely to surprise you later

THE TWO PROBLEMS IT SOLVES

The loader reports only the first hard error it finds before aborting — so you never see the shape of the problem. And it has no concept of a warning, so a valid config that quietly uses defaults you did not intend is accepted without comment.

One command, three ways to run it

Read-only, with no effect on the engine or any runtime state. Safe to run at any time — including while the exchange is live.

terminal

```
# the everyday check — a full human-readable report
pm-cverifier engine_config.yaml

# quieter: warnings and above only, no info-level advisories
pm-cverifier --level warn engine_config.yaml

# for CI: machine-readable, and fail the build on any warning
pm-cverifier --strict --format json engine_config.yaml
```

FLAG	WHAT IT DOES
--format	text (default) or json
--level	info, warn or error — the minimum severity to show
--strict	treat warnings as errors for the exit code
--no-color	disable ANSI colour in text output

Four layers, run in order

A hard failure in layer 1 or 2 stops the later layers — they are reported as skipped, because there is no point checking consistency in a file that does not parse.

LAYER 1

YAML syntax

Can the file be parsed at all?

File readable, valid YAML, top level is a mapping.

ERROR

LAYER 2

Schema

Do the fields exist and have the right types?

Required keys present, correct types, values in range.

ERROR

LAYER 3

Semantic

Are the values consistent with each other?

Cross-field rules and referential integrity between sections.

ERROR / WARN

LAYER 4

Completeness

Are recommended settings present and plausible?

Missing-but-recommended settings and default-value notices.

WARN / INFO

Within a layer, every independent problem is reported — so one pass surfaces as many issues as it possibly can.

Reading the report

Every finding carries a code, a severity, one sentence naming the exact field, and a concrete fix — often as a snippet you can paste.

text output

```
$ pm-cverifier engine_config.yaml
OK  YAML syntax: OK
OK  Schema: 0 errors
!   Semantic: 1 warning
i   Completeness: 2 advisories
-----------------------------------------------------
[M013] WARN  No gateway has role: ADMIN.
  -> Without an admin gateway, halt, resume, kill-switch
     and emergency commands cannot be issued at runtime.
     Add a gateway with role: ADMIN:
       - id: OPS01
         role: ADMIN
         disconnect_behaviour: LEAVE_ALL
```

Three severities

- **ERROR** the engine will not load this
- **WARN** it loads, but review it
- **INFO** a default you may not have intended

Exit codes

- **0** no errors, no warnings
- **1** warnings only
- **2** one or more hard errors
- **--strict** any warning also returns 2

PM-CVERIFIER

The Risk Summary — the part to actually read

Printed at the end of every report. It answers a question no error list can: what does this configuration actually do at runtime?

Risk Summary

```
Symbols           2  (AAPL, MSFT)
Gateways          2  (TRADER01: TRADER, MM01: MARKET_MAKER)
Sessions          disabled - always CONTINUOUS
Collars           enabled - DEFAULT: static=20%, dynamic=2%
Circuit breakers  enabled - L1=7% (5 min), L2=13% (15 min),
                  L3=20% (rest-of-day)
MM obligations    not enforced
Admin gateway     none  !
```

Why it is the most useful output

An error list tells you what is broken. The risk summary tells you what is true. It is the fastest way to catch a configuration that is perfectly valid and completely wrong for what you intended.

A marker beside a line flags a risky or incomplete subsystem even when the overall verdict is OK — a missing admin gateway, for instance, or absent collar configuration.

Read this out loud before every first start.

Ten mistakes it catches before you do

A sample from the catalogue. Each has a stable code you can look up, and most name the exact field.

M002	A seed quote names a gateway that does not exist	ERROR
M020	A seed quote names a gateway that is not a market maker	ERROR
S015	A seed quote's bid is at or above its ask — a crossed book on startup	ERROR
M004	Sessions are enabled but there is no schedule	ERROR
M021	A schedule time was written unquoted and parsed as a number	ERROR
M018	Two configured listeners would bind the same port	ERROR
S013	A symbol references a risk level that was never defined	ERROR
M009	An index constituent is not in the symbol universe	ERROR
M013	No gateway has the ADMIN role — nobody can halt the market	WARN
C001	No symbol has a reference price — collars have nothing to anchor to	WARN

The full catalogue runs to several hundred codes across four families — Y for syntax, S for schema, M for semantic, C for completeness.

Make it impossible to deploy a bad config

The verifier earns its keep when nobody has to remember to run it.

continuous integration

```
# pre-flight check inside a deployment script
pm-cverifier --strict --format json engine_config.yaml
echo "Config OK"

# as a build step
- name: Verify engine config
  run: pm-cverifier --strict --format json engine_config.yaml
```

WHERE IT SITS AMONG THE TOOLS

pm-config-gen and config-gui generate. pm-cverifier verifies. pm-config-deploy compiles and installs. pm-engine loads. Four tools, one file format.

Strict mode is the point

With --strict, any warning returns exit code 2. The build fails fast rather than silently shipping a market with no admin gateway and no collars.

JSON output is stable

The machine-readable form carries the same findings plus a risk_summary object with extra fields — including a flag telling you the built-in circuit-breaker ladder is in effect because you configured none.

SECTION 05

Why a compiled JSON, not the YAML spec file?

05

The single most important idea in this deck, and the one most likely to trip you up on day one: the file you edit is not the file that runs.

The drift problem

pm-config-deploy

The meta block

Two digests

THE PROBLEM

Eight loaders, eight copies of the truth

Every process parses the config independently. Before compilation, that meant every process also carried its own copy of every default.

What went wrong

- **Drift.** Eight independent default tables can disagree — and did.
- **No gate.** A configuration nobody had checked could reach a running exchange.
- **Ambiguity.** Different processes could be pointed at different files.
- **Invisibility.** An edit you forgot to apply looked identical to one you had.

What compilation fixes

- **One resolver.** Defaults are resolved exactly once, at compile time.
- **One gate.** All four verifier layers run before anything is installed.
- **One file.** No process accepts a config path, so they cannot diverge.
- **One fingerprint.** Processes warn at startup when the source has changed.

THE DESIGN SHIFT

Each process now deserialises rather than decides. It reads a fully-resolved section and applies it — there is no logic left in a loader for two loaders to disagree about.

THE SPLIT

The file you author is not the file that runs

Two artifacts, two jobs. Confusing them is the most common day-one mistake.

engine_config.yaml

THE FILE YOU AUTHOR

- **Lives** wherever suits you — normally in version control with the rest of your course material
- **Human** commented, diffable, reviewable in a pull request
- **Partial** you write only what differs from the defaults
- **Read by** you, your team, and the verification tools — never by a running process

> pm-config-deploy

engine_config.json

THE FILE THAT RUNS

- **Lives** at one fixed path inside the data directory, under ref_data
- **Machine** fully resolved, every default expanded, nothing to interpret
- **Complete** two authored keys compile to nine fully-populated sections
- **Read by** every running process — and by nothing else

THE CONSEQUENCE

No process accepts a config path. It is therefore not possible to start two of them against different files.

THE BRIDGE

pm-config-deploy does three things

It is the only route from an authored file to a running exchange — and the reason an unchecked configuration can no longer reach one.

1

Validates

Runs all four verifier layers against the authored file. A configuration nobody checked can no longer reach a running exchange.

2

Resolves

Expands every default exactly once, rather than in the eight loaders that used to hold their own copies and could drift apart.

3

Installs

Writes the result atomically, alongside a copy of the source it was built from — so the artifact always carries its own provenance.

terminal

```
pm-config-deploy my_config.yaml          # validate, compile and install
pm-config-deploy --check my_config.yaml  # validate only, install nothing
pm-config-deploy --show                  # where do the deployed files live?
```

DEPLOYING DOES NOT RESTART ANYTHING

Deployment replaces the running configuration but does not disturb live processes. Restart them to pick it up.

WHAT COMPILATION PRODUCES

Two keys in, nine resolved sections out

The artifact is not a reformatted copy of your YAML. It is the complete, expanded state of the exchange.

what you wrote

```yaml
symbols:
  AAPL:
    tick_decimals: 2
gateways:
  alf:
    - id: TRADER01
```

compiles to

- symbols
- gateways
- engine_flags
- risk_controls
- circuit_breaker_defaults
- mm_obligation_defaults
- schedule
- market_data_gateway
- …and every other block

THE POINT OF THE EXPANSION

A market_data_gateway block you never wrote still arrives with all of its fields populated. That is what lets each process deserialise a section rather than decide what it should have been.

SO WHY NOT JUST READ THE YAML?

Because your YAML is incomplete by design. Somebody has to fill in the other ninety percent — and if that somebody is eight separate loaders, they will eventually disagree.

PROVENANCE

The artifact records what it is

Every compiled file carries a meta block describing where it came from and what built it.

engine_config.json

```
"meta": {
  "schema_version": 3,
  "compiler_version": "0.17.0",
  "compiled_at": "2026-07-30T16:43:18.000Z",
  "source_path": "/Users/you/course/engine_config.yaml",
  "source_sha256": "e3bc2f14cf5c10da...",
  "content_sha256": "87be4dc0b9b645cb..."
}
```

source_sha256

Has the authored file changed since this was built?

Each process warns at startup when it has — so an edit you forgot to deploy is visible rather than silently ignored.

content_sha256

Has this file changed since it was built?

Recomputed on every load. Editing the deployed artifact by hand is refused, naming pm-config-deploy as the proper way to make the change.

TWO HONEST LIMITS

The payload digest detects modification, not malice — it travels inside the file it protects, so anyone who edits the payload can recompute it. And schema_version guards against a build reading an artifact shaped for another: an unknown version is refused with a message telling you to recompile.

SECTION 05 IN ONE SLIDE

Five reasons the JSON is the reference

If you remember nothing else from this section, remember these.

1	**Because it is validated**	Nothing reaches a running exchange without passing all four verifier layers first.
2	**Because it is complete**	Every default is resolved exactly once, at compile time, by one piece of code.
3	**Because it is singular**	No process accepts a config path. Two processes cannot be run against different files.
4	**Because it is fingerprinted**	Processes warn when the authored source has moved on, and refuse a hand-edited artifact.
5	**Because it is versioned**	A build refuses an artifact shaped for a different schema rather than misreading it.

The YAML remains the thing you edit, review and version. It just is not the thing that runs.

SECTION 06

The top-level sections, explained

06

One slide per section: what it is for, its most important fields, the defaults you inherit if you say nothing, and the mistake people make.

Universe & access

Risk policy

Liquidity

Rhythm & plumbing

SECTION REFERENCE

symbols

REQUIRED

A mapping of symbol name to its settings. This is the instrument allowlist — an order for anything not listed is rejected outright.

FIELD	DEFAULT	WHAT IT DOES AND WHY IT MATTERS
`tick_decimals`	2	Price precision, 0 to 8. Two means prices move in cents. This fixes the price grid everything else snaps to.
`last_buy_price`	—	A seed reference price. On day one there is no trade history, so a collar has nothing to anchor to without it.
`last_sell_price`	—	The other half of the day-one reference. Set both, and set them consistently with your opening quote.
`outstanding_shares`	—	Share count. Optional until the symbol joins an index, where it becomes required to compute the weight.
`level`	—	Names a risk level from risk_controls. The level supplies this symbol's collar bands.
`collar`	—	A per-symbol band override, merged over the level's collar. Symbol keys win.
`market_maker_quotes`	—	Opening bid and ask injected when the named market-maker gateway authenticates.
`circuit_breaker`	—	A per-symbol override of the halt ladder, merged level by level over the exchange defaults.

WATCH OUT

Symbol names are upper-cased on load, so aapl and AAPL are the same instrument. A symbol with an empty settings block is perfectly valid — it simply inherits everything.

SECTION REFERENCE

gateways.alf

REQUIRED

A list with at least one entry. This is the participant allowlist: who may connect, as what, and what happens to their orders when they disappear.

FIELD	DEFAULT	WHAT IT DOES AND WHY IT MATTERS
`id`	—	The gateway identity. Must exactly match what the client passes on the command line. Upper-cased, and must be unique.
`description`	empty	Free text, shown in the operator console's gateway table. Use it — future you will be grateful.
`role`	TRADER	TRADER, MARKET_MAKER or ADMIN. Determines what commands the session may issue.
`disconnect_behaviour`	CANCEL_QUOTES_ONLY	What happens to resting orders when the session drops: cancel quotes only, cancel everything, or leave everything.
`quote_refresh_policy`	INACTIVATE_ON_ANY_FILL	When a seeded quote stops being live after a fill — on any fill, on a full fill, or never.
`smp_action`	NONE	Self-match prevention for this gateway's orders when the order itself does not specify one.
`mm_obligations`	—	Per-symbol market-making duties for this gateway — the finest-grained level of obligation control.

WATCH OUT

No gateway ID may be a prefix of another. TRADER0 and TRADER01 together will be rejected — the gateways cannot tell the sessions apart.

The three roles, and what they unlock

Role is not about trust. It is about which commands the engine will accept from that session.

TRADER

The default. Submits, amends and cancels its own orders. Everything a participant in a classroom exercise needs.

Nothing special. This is the baseline.

MARKET_MAKER

Everything a trader can do, plus two-sided quoting. Seed quotes are injected on its behalf when it authenticates.

Unlocks quote seeding and obligation enforcement.

ADMIN

Everything a trader can do, plus the five exchange-wide controls: halt, resume, per-symbol halt and resume, and symbol-level mass cancel.

Without one, nobody can stop the market.

TWO THINGS BEGINNERS GET WRONG HERE

Connecting does not require ADMIN — any configured ID can open an operator console and run read-only commands. The role is checked per command, and only for those five. And adding a MARKET_MAKER gateway is what makes seed quotes mandatory on every symbol: the role creates the obligation.

SECTION REFERENCE

Engine behaviour flags

Four global switches. All default to the safe answer, which is not always the answer you want.

sessions_enabled

default: true

When true the engine starts CLOSED and honours session transitions. When false it starts and stays in CONTINUOUS.

This is the number one reason a new exchange "does not match anything". It is true by default, so without a scheduler the market never opens.

enforce_collars

default: true

The master switch for price-band rejection across the whole exchange.

Turning it off does not delete your collar settings. It just stops applying them — and the verifier will tell you so.

enforce_circuit_breakers

default: true

The master switch for automatic volatility halts.

Leave it on. A halted symbol is a teaching moment; a symbol that fell ninety percent is an incident.

snapshot_interval_sec

default: 0.5

The minimum gap between published order-book snapshots for a busy symbol.

Lower means more data and more load. Below 0.2 across more than twenty symbols draws a warning.

SECTION REFERENCE

risk_controls — how a collar works

A collar is a band around a reference price. An order priced outside it is rejected before it can ever trade.

- **static_band_pct** default 0.20 — anchored to a session reference price
- **dynamic_band_pct** default 0.02 — tracks near-live prices as they move
- **levels** named bundles of collar settings that symbols reference by name
- **default_level** applied to any symbol that does not name its own

THE MERGE RULE

A per-symbol collar is merged over the symbol's resolved level collar, and symbol keys win. If neither is present, no collar applies to that symbol at all — which is a silence, not an error.

WHY A BEGINNER NEEDS THIS ON DAY ONE

A student who types 15000 instead of 150.00 prints a trade at a hundred times the market. The collar is the difference between a typo and a ruined exercise.

SECTION REFERENCE

circuit_breaker_defaults — the halt ladder

When a price moves too far, too fast, trading in that symbol stops for a cool-off. The ladder decides how far is too far.

L1

7%

halt for 5 minutes

A sharp move. Pause, let everyone re-read the book.

L2

13%

halt for 15 minutes

A serious dislocation. A longer pause.

L3

20%

halt for rest of day

Something is wrong. The symbol is done for the day.

This is the built-in ladder. It applies if you configure nothing at all.

- **reference_window_ns** the rolling window the price shift is measured against — five minutes by default
- **price_shift_pct** how far the price must move to trigger this level, as a fraction between 0 and 1
- **halt_duration_ns** the cool-off. Leave it empty to halt for the remainder of the trading day

Reopening — Automated Corridor Expansion

A halted symbol does not simply resume. It reopens through a corridor that starts narrow — ten percent by default — and widens in timed stages until the market can cross again. A small random tail on the end prevents everyone timing the reopen to the second.

SECTION REFERENCE

mm_obligation_defaults — three tiers of inheritance

A market maker promises to keep a tight, sized, two-sided quote. This is where you say how tight, how big, and whether you actually check.

Exchange-wide defaults

`mm_obligation_defaults`

enforce (default off) · max spread 10 ticks · min quantity 100

Per-symbol override

`mm_obligation_defaults.symbols.<SYM>`

Relax or tighten the duty for one instrument — a thin small-cap need not quote like a blue chip

Per-gateway, per-symbol

`gateways.alf[].mm_obligations.<SYM>`

The finest grain: this market maker, this instrument. Highest precedence of all

higher precedence ↓

THE THREE SETTINGS, IN PLAIN ENGLISH

Is the duty checked at all · how wide may the bid-to-ask spread be, counted in ticks · how much size must sit on each side. A quote that breaks the promise is rejected.

SECTION REFERENCE

Seeding the book: quotes, combos and indices

Three optional sections that make an empty exchange feel like a real market from the first second.

LIQUIDITY

market_maker_quotes

A per-symbol list of opening quotes, each attributed to a market-maker gateway.

bid_price must be strictly below ask_price, and both sizes must be positive — a crossed seed quote is rejected.

seed_once, on by default, means the engine skips injection if it already holds book state for that symbol. A restart with history does not double-seed.

The quote is injected when the named gateway authenticates, not at engine startup.

ADVANCED

market_maker_combos

Multi-leg seed orders: between two and ten legs, placed as a single all-or-none package.

Each leg names a symbol, a side, an order type and a quantity. Symbols must be unique within a combo and must exist in the universe.

Use them to teach spreads, pairs and hedged entries — where the whole package trades or none of it does.

An expert-level feature. Skip it until the basics are solid.

ANALYTICS

indices

Up to five calculated, cap-weighted indices.

Each names its constituents, a base value (default 1000) and a publish interval (default one second).

Every constituent must be a configured symbol that also defines outstanding_shares — without a share count there is no weight to compute.

The index process reads this section and the per-symbol share counts, and nothing else.

SECTION REFERENCE

schedule and country — the trading day

Five wall-clock times that must be strictly increasing, plus a holiday calendar.

09:00 — PRE_OPEN
Orders accepted, nothing matches

09:25 — OPENING AUCTION
The uncross that sets the open

09:30 — CONTINUOUS
Normal trading, price-time priority

16:00 — CLOSING AUCTION
The uncross that sets the close

16:05 — CLOSED
The day ends

QUOTE YOUR TIMES

Write "09:30" with quotes. Unquoted, YAML reads it as a sexagesimal number and your schedule silently becomes nonsense. The verifier catches this one.

COUNTRY IS A SIBLING, NOT A CHILD

It sits at the top level next to schedule — not inside it. Default "Sweden". The scheduler uses it to skip weekends and bank holidays.

THE ONE FORGIVING FIELD IN THE WHOLE FILE

An unrecognised country does not abort the load. The scheduler logs a warning and falls back to the default. Every other invalid value in the file is fatal.

SECTION REFERENCE

The seven auxiliary gateway blocks

Learn one and you have learned all seven — they share the same shape. Each is read only by its own process.

BLOCK	PROCESS	PORT	WHAT IT SERVES
alf_gateway	pm-alf-gwy	5565	Text order entry. Also reads gateways.alf for session identity and role.
balf_gateway	pm-balf-gwy	5560	Binary order entry. Adds a duplicate-session policy: reject the new one, or evict the old.
market_data_gateway	pm-md-gwy	5570	The market-data feed. Carries a replay window and the number of depth levels to publish.
post_trade_gateway	pm-ralf-gwy	5580	Fills for clearing, drop-copy and audit subscribers, with an allowed-roles list.
dc_gateway	pm-dc-gwy	5590	A drop-copy relay. Has no enabled key — configuring it is enabling it.
api_gateways	pm-api-gwy	8080	Named REST and WebSocket instances, each with its own API credentials and rate limits.
log_server	pm-log-srv	5600	The central log collector, with retention, batching and a log-distribution interface.

TWO RULES THAT CATCH PEOPLE OUT

The API block is plural — api_gateways — because it holds named instances. The singular form is rejected outright. And post_trade_gateway and dc_gateway have no enabled switch at all: if the block is there, the service is on.

SECTION 07

What is a minimal configuration?

07

Two answers, and the gap between them is where beginners get stuck: the smallest file that loads, versus the smallest file that gives you something worth trading on.

The spec minimum

The gap

The practical minimum

A checklist

ANSWER ONE

The smallest file that loads

Straight from the normative specification. Two sections, four lines of content.

the minimal conformant document

```
gateways:
  alf:
    - id: TRADER01
symbols:
  AAPL:
    tick_decimals: 2
    last_buy_price: 149.90
    last_sell_price: 150.10
```

What you get

- **One instrument** AAPL, priced in cents around 150
- **One participant** TRADER01 may connect
- **Default risk** collars and circuit breakers on, at their built-in values
- **Sessions on** so the engine starts CLOSED and waits

AND ONE CONDITIONAL RULE

Add a gateway with the MARKET_MAKER role and the specification immediately requires a market_maker_quotes block on every symbol. The role creates the obligation.

THE GAP

Why the minimum is not enough

That file loads perfectly. Then you connect, and nothing happens. Here is why.

The market is closed

Add a schedule, or set sessions_enabled to false.

sessions_enabled defaults to true, so the engine starts CLOSED and waits to be driven through the day. With no schedule and no scheduler, it waits forever.

The book is empty

Add a MARKET_MAKER gateway and a seed quote.

There is no market maker and no seed quote, so there is nothing to trade against. The first order just rests there alone.

Nobody can intervene

Add a gateway with the ADMIN role.

No gateway has the ADMIN role, so nothing can halt or resume the market, or mass-cancel a symbol, if something goes wrong.

One participant is not a market

Add a second trader.

One trader can only trade with themselves — and self-match prevention may well stop even that.

None of these is an error. All four are silences — which is exactly what makes them hard for a beginner to diagnose.

ANSWER TWO

The minimum that is actually useful

engine_config.yaml

```yaml
gateways:
  alf:
    - id: TRADER01
      description: First trader
    - id: MM01
      description: Market maker
      role: MARKET_MAKER
    - id: GW_ADMIN
      description: Operator console
      role: ADMIN
symbols:
  AAPL:
    tick_decimals: 2
    last_buy_price: 149.90
    last_sell_price: 150.10
    market_maker_quotes:
      - gateway_id: MM01
        bid_price: 149.90
        ask_price: 150.10
        bid_qty: 500
        ask_qty: 500
```

- **Three roles, one file.** A trader to act, a market maker to quote against, an operator to intervene.
- **A live book from second one.** The seed quote is injected when MM01 authenticates, so there is always something to hit.
- **Prices that agree.** The seed quote sits exactly on the last buy and sell prices, so the collar reference and the visible market are the same number.
- **Room to grow.** Add symbols by copying the AAPL block. Add participants by adding a line.

Or generate exactly this: `pm-config-gen --symbols AAPL --gateways TRADER01 MM01:MARKET_MAKER GW_ADMIN:ADMIN --output engine_config.yaml`

BEFORE THE FIRST START

The seven-point checklist

Read this list out loud against your file. Each line has a reason.

✓ **At least one entry under gateways.alf**	No gateways means nobody can connect at all.
✓ **At least one entry under symbols**	No symbols means no books, and nothing to trade.
✓ **Every gateway ID matches what the client will pass**	IDs are matched case-insensitively on connect, but a mismatch is silent — the client just times out.
✓ **At least one gateway has the ADMIN role**	Required to halt, resume, or mass-cancel from the operator console.
✓ **tick_decimals is correct for each symbol**	It fixes the price grid. Wrong here means every price is wrong.
✓ **Sessions decided: a schedule, or sessions_enabled false**	Omit both and the market never opens.
✓ **Each symbol that needs liquidity has a seed quote**	Referencing a MARKET_MAKER gateway. Without it the book starts empty.

SECTION 08

What processes read from the configuration?

08

One shared file, ten readers, and not one of them reads all of it. Understanding this split explains a whole class of confusing failures.

The ownership rule

The reader map

Indirect readers

Two consequences

THE OWNERSHIP RULE

No single process reads the whole file

Each process opens the configuration independently at its own startup, ignores every key it does not recognise, and parses only the sections it owns.

10
processes read from it

1
reads most of it

0
read all of it

THE ENGINE IS THE BIGGEST READER — AND STILL NOT THE ONLY ONE

pm-engine reads the symbol universe, the gateway allowlist, the behaviour flags, risk controls, circuit breakers, market-maker obligations, combos, indices and the schedule. It never touches a single one of the seven gateway blocks.

Permissive by design

A key a loader does not define is ignored, not rejected. That is what lets ten processes share one file — but it also means a typo in a key name is simply invisible to the process reading it.

The one exception

pm-cverifier is a static linter, not an operational reader. It parses and cross-validates the whole file — including sections that no running process consumes on its own.

THE READER MAP

Who reads what

Direct readers. Find your process, and you know which part of the file can possibly affect it.

PROCESS	SECTIONS IT READS	WHAT IT NEEDS THEM FOR
pm-engine	symbols · gateways.alf · behaviour flags · risk_controls · circuit_breaker_defaults · mm_obligation_defaults · market_maker_combos · schedule · indices	The market itself: universe, access, session and risk policy, obligations, startup seeds
pm-alf-gwy	alf_gateway · gateways.alf	Its own bind address, port and timeouts, plus the gateway allowlist and roles
pm-balf-gwy	balf_gateway · gateways.alf	The same, plus each gateway's disconnect behaviour for binary sessions
pm-md-gwy	market_data_gateway	Bind settings, the replay window and how many depth levels to publish
pm-ralf-gwy	post_trade_gateway	Bind settings and the roles allowed to subscribe to post-trade data
pm-dc-gwy	dc_gateway	Bind settings and the per-client queue limit for the drop-copy relay
pm-api-gwy	api_gateways	Named REST and WebSocket instances, credentials, rate limits and timeouts
pm-log-srv	log_server	Bind, retention and throughput settings for the central log collector

THE READER MAP

Indirect readers, and why they matter

Two processes read the config through another process's loader, and one reads it without running anything at all.

INDIRECT

pm-index

Reads the index definitions plus, for every constituent, the outstanding share count and the reference prices.

It wraps the engine's own loader rather than defining its own — which is why an index constituent missing a share count is an error the engine would never have raised on its own.

INDEPENDENT

pm-scheduler

Reads only the schedule and the country.

It re-reads the same schedule block the engine reads, but as an independent process — so the trading day can be driven or tested from outside. The country is used to skip weekends and bank holidays.

EVERYTHING

pm-cverifier

Reads everything.

As a static linter it parses and cross-validates the whole file, including sections that no runtime process consumes on its own. It is the only thing that ever sees the complete picture before deployment.

CONSEQUENCE ONE — A TYPO CAN HIDE IN PLAIN SIGHT

A mistake in the binary gateway block will not be caught by the engine at all. Only that gateway's own process, or the verifier, will ever reject it. Run the verifier before starting a full stack.

CONSEQUENCE TWO — YOU CAN RESTART JUST ONE PIECE

Change the market-data depth level and restart only that gateway. The engine never reads that section, so it does not need to know.

SECTION 09

A workflow for setting up a new exchange

09

Nine steps, in the order that avoids rework. Follow them once and the tenth exchange takes twenty minutes.

Decide

Author

Verify

Prepare

Deploy

END TO END

Nine steps, in order

Each step is cheap. Doing them out of order is what costs an afternoon.

1 Decide the shape

How many instruments, how many participants, is there a market maker, does the day have sessions

2 Generate a skeleton

pm-config-gen or the config GUI. Do not start from a blank file

3 Set the universe

Symbols with tick precision and honest reference prices — the anchor everything else uses

4 Set access

One gateway per participant, plus a market maker and an operator with the ADMIN role

5 Seed the book

A two-sided quote per symbol, priced to agree with the reference prices

6 Decide the day

A schedule with quoted times, or sessions_enabled set to false for always-open

7 Verify

pm-cverifier. Fix every error, read every warning, then read the risk summary out loud

8 Deploy

pm-config-deploy. Nothing runs against your YAML — this is the step people forget

9 Start and prove

Engine first. Then confirm the running exchange lists the symbols and gateways you intended

Steps 7 and 8 are the two that beginners skip, and they are the two that cause the most confusion later.

STEP 9

Startup order: the engine goes first

The engine binds the sockets everything else connects to. A process that starts too early either times out or silently drops its first messages.

FIRST

pm-engine

Binds all three sockets and loads the deployed configuration. Wait for its listening line.

THEN

pm-scheduler

Optional. Must connect before the first scheduled time or it misses that transition.

THEN

Gateways

One console per participant. The ID must match a configured gateway exactly.

THEN

Observers

Viewers, order monitors, audit, clearing and statistics — in any order.

LAST

Displays

Ticker and board read the statistics database, so statistics must be running first.

HOW DO YOU KNOW THE ENGINE IS READY?

Run it verbosely and wait for the line that reports the bound sockets. Without that flag the engine starts identically but prints nothing — those are informational messages, and the default level is higher.

RESTARTING ONE PROCESS IS SAFE

Every process connects to the engine, never the other way round. Any non-engine process can be stopped and restarted at will. Restarting the engine, however, disconnects every gateway.

TIMING

A conventional stagger of a third of a second to a second between processes is enough.

STEP 9

Prove the configuration actually took effect

"The engine started" is not proof. These four checks are.

1 Read the startup banner

It names the loaded config, the symbol count, the gateway count, and whether sessions and risk enforcement are on. If the counts are not what you expect, stop here.

2 Ask the running exchange

From an operator console, list the symbols and the gateways. This is the engine telling you what it believes, not what your file says.

3 Check the session state

Query the current session state and the schedule. A market stuck in CLOSED with no scheduler running is the most common first-day surprise.

4 Send one order

A single limit order proves the whole path: client to engine, matching, and the event back out again. Then watch it appear in a book viewer.

STEP 9

Symptom to config mistake, in one table

Almost every startup problem on a new exchange is one of these six.

WHAT YOU SEE	WHAT IT REALLY IS	THE FIX
Fatal: gateways.alf must be a list	You wrote it as a mapping, without dashes	Correct the shape of the section
Fatal: symbol config must be a mapping	A symbol entry is a list or a scalar	Correct the shape of that symbol
Gateway authentication timed out	The ID is not in the allowlist — the engine ignores it silently	Add the ID, then restart the engine
Order rejected: symbol not configured	The symbol is absent from the universe	Add it to symbols and restart
Scheduler exits: config missing	There is no schedule block at all	Add a schedule, or run it in immediate mode
Halt or resume is rejected	The gateway exists but has no ADMIN role	Add the role, then restart the engine
The book is empty and nothing matches	No seed quote, or the market is still CLOSED	Add a seed quote; check the session state

THE PATTERN

Five of these seven are silences rather than errors. That is precisely why the verifier and the risk summary are worth the thirty seconds they cost.

AFTERWARDS

Changing a configuration safely

The same loop, every time, whether you are adding one symbol or rebuilding the risk policy.

1

Edit the YAML

The authored file, in version control. Never the deployed artifact — that edit is refused.

>

2

Verify

Fix every error and read every warning before going further.

>

3

Deploy

Compile and install. Live processes are not disturbed by this.

>

4

Restart what changed

Only the processes that read the sections you touched.

ADDING OR REMOVING A SYMBOL

The universe is built once at engine startup. A new symbol needs an engine restart before an order for it will be accepted.

TUNING ONE GATEWAY

Change a gateway's own block and restart only that gateway. The engine never reads those sections, so it does not need to know.

THE HABIT WORTH BUILDING

Treat the authored file like source code: reviewed, versioned, and never edited on a live machine. The deployed artifact is a build output, not a place to make changes.

IN CLOSING

Eight rules worth remembering

01

Two sections are required. Everything else has a default — know what you are inheriting.

02

A missing config file is not an error. It is unrestricted mode, and it is not what you want.

03

You author YAML. Nothing runs it. Deploy, or your change does not exist.

04

Verify before you deploy, and read the risk summary out loud before you start.

05

Quote your schedule times. Unquoted, they silently become numbers.

06

Sessions are on by default, so a new engine starts closed and waits.

07

No single process reads the whole file — which is why a typo can hide from the engine.

08

The engine starts first. Everything else connects to it, never the other way round.

The configuration is the exchange. Treat it like the most important file you own.

SECTION R

Technical reference

R

Tables to come back to. Nothing new here — this is the material from the previous sections, arranged for lookup rather than for learning.

Enumerations

Constraints

Defaults

Commands

Enumerations

Every enum value is case-insensitive on input and stored upper-case. A value outside the member set is rejected.

ENUM	MEMBERS	USED BY
Role	TRADER · MARKET_MAKER · ADMIN	gateway role
DisconnectBehaviour	CANCEL_QUOTES_ONLY · CANCEL_ALL · LEAVE_ALL	gateway disconnect
QuoteRefreshPolicy	INACTIVATE_ON_ANY_FILL · INACTIVATE_ON_FULL_FILL · NEVER_INACTIVATE	gateway quoting
TIF	DAY · GTC · ATO · ATC	quote and combo seeds
ComboType	AON	combo seeds
Side	BUY · SELL	combo legs
OrderType	MARKET · LIMIT · STOP · STOP_LIMIT · FOK · ICEBERG · IOC · TRAILING_STOP	combo legs
SmpAction	NONE · CANCEL_AGGRESSOR · CANCEL_RESTING · CANCEL_BOTH	self-match prevention
DuplicateSessionPolicy	REJECT_NEW · EVICT_OLD	binary gateway sessions

REFERENCE

Cross-field constraints

Rules that span more than one section. A violation is rejected at load — with one documented exception.

- Both required sections present; the gateway list has at least one entry
- If any gateway is a market maker, every symbol must define at least one seed quote
- Every seed quote has a bid strictly below its ask, and positive sizes on both sides
- A circuit-breaker block may not appear inside a risk level
- At most five indices; every constituent exists and defines a share count
- Tick decimals between zero and eight; share counts positive when present
- No two configured listeners resolve to the same effective port
- Gateway IDs are unique, and no ID is a prefix of another
- Every seed quote names a gateway that exists and carries the MARKET_MAKER role
- Every risk level a symbol references — explicitly or by default — actually exists
- Each combo has two to ten legs, with unique symbols that all exist in the universe
- Collar bands and price shifts are fractions strictly between zero and one
- The singular API gateway key is not supported, and credentials are not shared across instances
- An unrecognised country is the sole exception: it warns and falls back to the default

REFERENCE

Defaults you inherit by saying nothing

If a field is not in your file, this is what the exchange uses.

FIELD	DEFAULT
tick_decimals	2
sessions_enabled	true
enforce_collars	true
enforce_circuit_breakers	true
snapshot_interval_sec	0.5
role	TRADER
disconnect_behaviour	CANCEL_QUOTES_ONLY
smp_action	NONE
country	Sweden

FIELD	DEFAULT
static_band_pct	0.20
dynamic_band_pct	0.02
CB ladder L1 / L2 / L3	7% / 13% / 20%
CB halt durations	5 min / 15 min / day
reference window	5 minutes
mm_max_spread_ticks	10
mm_min_qty	100
enforce_mm_obligation	false
index base value	1000.0

THE TWO TO DOUBLE-CHECK

Sessions default to on, so the engine starts closed. Market-maker obligations default to off, so nobody is being graded on quote quality unless you say so.

REFERENCE

Command cheat sheet

The commands from this deck, in the order you would actually use them.

the whole lifecycle

```
# 1 — generate a starting point
pm-config-gen --symbols AAPL MSFT --gateways TRADER01 MM01:MARKET_MAKER GW_ADMIN:ADMIN \
              --output engine_config.yaml

# 2 — verify it
pm-cverifier engine_config.yaml                          # full report
pm-cverifier --level warn engine_config.yaml             # warnings and above
pm-cverifier --strict --format json engine_config.yaml   # for CI

# 3 — compile and install it
pm-config-deploy engine_config.yaml                      # validate, compile, install
pm-config-deploy --check engine_config.yaml              # validate only
pm-config-deploy --show                                  # where do the files live?

# 4 — start it, engine first
pm-engine -v
pm-scheduler                                             # or --now for testing
pm-alf-console --id TRADER01
pm-admin --id GW_ADMIN
```

REFERENCE

Where to go next

This deck is a guided tour. These are the documents it was built from.

990-app-config-spec.md The normative reference	The formal schema: every field, type, default and constraint. Where this deck and that document disagree, that document wins.
010-configuration.md The full configuration guide	Worked examples from minimal to fully complex, the process-to-section map, and a section per domain topic.
020-config-verifier.md The verifier	The complete CLI reference, all four layers, and the full catalogue of check codes with their meanings.
030-config-GUI.md The Config Builder	Running it, the personas, every tab, the workflows, and troubleshooting.
040-running-the-exchange.md Running the exchange	Startup order, the full process reference, verification steps and startup troubleshooting.

Start with the guide, keep the specification open beside it, and run the verifier after every change.